PROJECT REPORT

Module 6 - In-Vehicle Infotainment Systems Message Flow Simulator

Submitted by

Name	:	Shraman Dasgupta
Department	:	CSE
Roll No.	:	4C-56
College	:	IEM, New Town

Table of Contents

Abstract 2

1. Problem Definition 2

2. Objectives 2

3. Functional Architecture 2

4. Component Design 3

5. Implementation Details 3

6. Scenarios and Generated Outputs 3

7. Sample Trace 4

8. Test Strategy 4

9. Limitations 4

10. Future Scope 4

11. Conclusion 5

12. References 5

Abstract

This project implements a software simulation of an IVI audio control path from user interaction to the Audio HAL. The system models Media Service, Audio Service, AudioFlinger, Audio HAL and a downstream DSP/amplifier boundary. Each transition is captured as an Android-style log entry and a structured CSV message trace. Three repeatable scenarios - normal playback, navigation ducking and incoming-call interruption - provide a practical demonstration of control propagation and audio-state changes.

1. Problem Definition

The Module 6 brief asks for a simulation of message flow between Media Service and Audio HAL and a demonstration of how user input propagates through the stack. The project therefore prioritizes traceability, repeatability and a clear separation between software components.

2. Objectives

- Model the IVI media-to-audio message path in a readable software architecture.
- Generate Android-style logcat messages for each important transition.
- Produce a structured message trace and machine-readable run summary.
- Validate the flow with repeatable scenarios and automated tests.

3. Functional Architecture

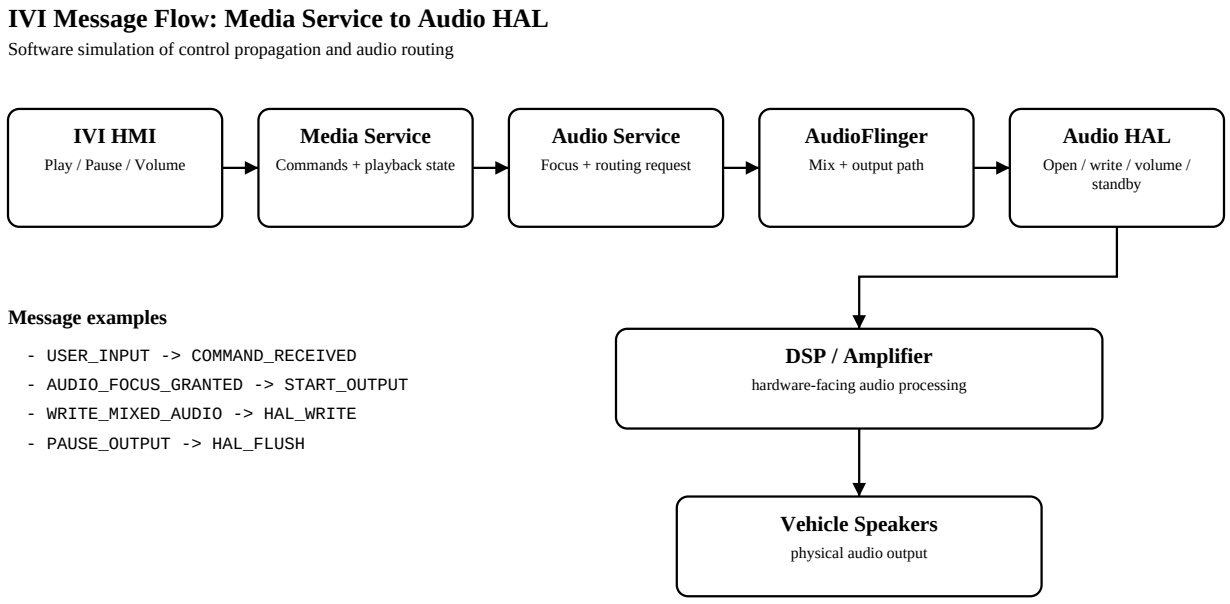

Figure 1. Implemented functional architecture.

4. Component Design

Component	State / data	Key operations
IVI HMI	User command + payload	user_input()
MediaService	IDLE / PLAYING / PAUSED + current track	PLAY, PAUSE, RESUME, NEXT, STOP, VOLUME, DUCK
AudioService	Focus holder	request_focus(), start(), pause(), set_volume()
AudioFlinger	Simplified routing/mix stage	start_output(), write_audio(), pause_output()
AudioHAL	CLOSED / OPEN / STANDBY + volume	open(), write(), flush(), set_volume(), standby()
TraceLogger	Message list + log lines	emit()

5. Implementation Details

The simulator is intentionally dependency-light: the runtime code uses only the Python standard library. This reduces setup complexity and makes the demonstration portable. The internal representation uses a Message data class containing sequence number, timestamp, source, destination, event and payload.

The TraceLogger emits two synchronized representations: a human-readable Android-style line and a structured Message record. The same records are written to CSV for inspection and summarized in JSON for automated checks.

6. Scenarios and Generated Outputs

Scenario	Messages	HAL commands	Focus grants
normal	39	7	2
navigation	26	5	1
call	32	5	2

The message counts above are generated directly from the submitted simulator and therefore vary only if the source code or scenario definition changes.

7. Sample Trace

```
18:58:39.819 I/IVI_HMI: -> MediaService | USER_INPUT | command=PLAY track=Connected Drive
18:58:39.819 I/MediaService: -> MediaService | COMMAND_RECEIVED | command=PLAY track=Connected Drive
18:58:39.819 I/AudioService: -> AudioFlinger | AUDIO_FOCUS_GRANTED | owner=MediaService
18:58:39.819 I/AudioService: -> AudioFlinger | START_OUTPUT
18:58:39.819 I/AudioFlinger: -> AudioHAL | OPEN_OUTPUT
18:58:39.819 I/AudioHAL: -> DSP/Amplifier | HAL_OPEN | sample_rate=48000 channels=2
18:58:39.819 I/AudioFlinger: -> AudioHAL | WRITE_MIXED_AUDIO | frames=1024
18:58:39.819 I/AudioHAL: -> DSP/Amplifier | HAL_WRITE | frames=1024
18:58:39.819 I/MediaService: -> IVI_HMI | STATE_UPDATE | state=PLAYING track=Connected Drive
18:58:39.819 I/TelephonyService: -> AudioService | AUDIO_FOCUS_LOSS | reason=incoming_call
```

Figure 2. Incoming-call interruption trace (excerpt).

8. Test Strategy

Test	Expected result	Status
Normal playback creates HAL_OPEN and HAL_WRITE	HAL commands appear after START_OUTPUT	PASS
Call interruption creates focus-loss and resume events	AUDIO_FOCUS_LOSS, CALL_ENDED and RESUME_OUTPUT exist	PASS
Saved summary matches trace length	JSON message_count equals in-memory count	PASS

9. Limitations

- The project simulates the control/message semantics; it does not run inside Android Automotive OS.
- The Audio HAL is modeled as a stateful software component and does not access a physical audio device.
- The audio mixer is simplified and intended for educational traceability rather than signal-quality measurement.

10. Future Scope

- Port the same state transitions into an Android Automotive test application using Media3 / MediaSessionService.
- Capture real logcat output from an emulator and compare it with the simulated trace.
- Add formal UML sequence/state diagrams and timing measurements for each message path.

11. Conclusion

The implemented simulator demonstrates the assignment requirement by making the message flow from IVI user input to Media Service and then to the Audio HAL explicit, repeatable and measurable. It provides both visual documentation and executable code, while keeping the scope aligned with the supplied Module 6 option.

12. References

[1] Android Open Source Project. “Automotive audio”.
https://source.android.com/docs/automotive/audio

[2] Android Developers. “Manage audio focus”.
https://developer.android.com/media/optimize/audiofocus

[3] Android Developers. “Background playback with a MediaSessionService”.
https://developer.android.com/media/media3/session/background-playback